

Software Database: first-week progress

Purpose

Give ArchEvolve a shared record of computation, memory behavior, and measured performance.

Implemented	Current coverage
Describe and query application code	6 kernels, 8 implementations
Profile each implementation on each input	32 profiles across 4 inputs
Screen optimization strategies	5 strategies, 2 intrinsics

31 profiles complete. Strategy application and measured gains are the next step.

One kernel can have different memory behavior

Example: PageRank reads a neighbor's contribution

```
contribution[neighbor_index[e]]
```

Simplified expression: stream through neighbor_index, then read contribution indirectly.

Recorded fact	Gauss–Seidel	Jacobi
Kernel (shared correctness check)	PageRank	PageRank
Access pattern's update kind	Read	Read
Loop-carried dependency	Yes	No

Code semantics belong to the implementation. Measurements belong to its profiles.

Strategy queries use those code semantics

Packing copies gathered values into a contiguous array

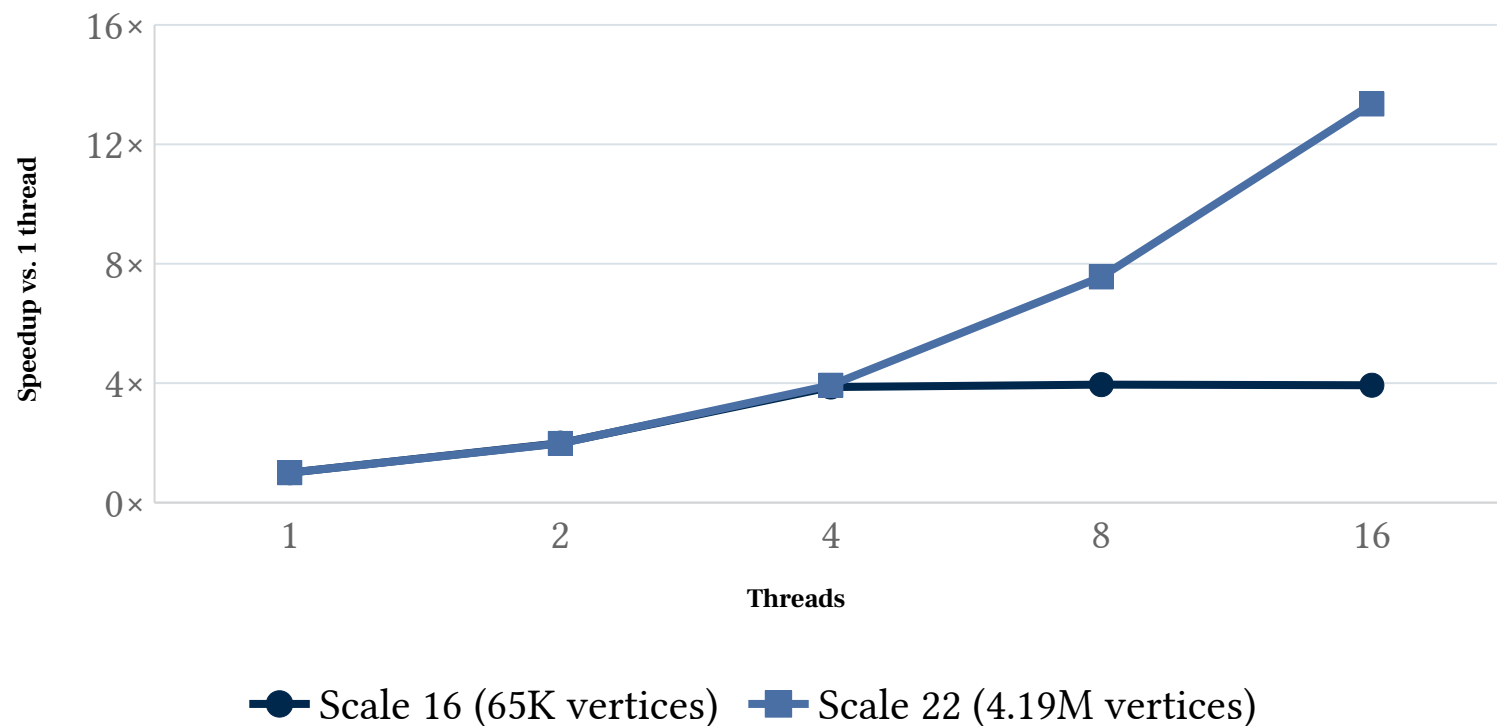
Strategy	Gauss–Seidel	Jacobi
Packing	Illegal	Legal
SIMD gather	Illegal	Legal
Software prefetch	Legal	Legal

“Legal” means encoded conditions pass. Manual checks still remain.
Unknown required facts produce “undetermined.”

Strategy screening and instruction-set checks work. No measured strategy gains yet.

PageRank scaling changes with input size

Jacobi PageRank on mbit10: median of 5 trials per thread count



At 16 threads

Scale 16: 3.93x

Scale 22: 13.35x

Footprint / socket L3

Scale 16: 0.33x

Scale 22: 23.06x

Same implementation, different scaling. Profiles must retain input and machine context.

Next step: apply and evaluate one strategy

Completed

Records, queries, profiling, workload export, and strategy screening

Current limits

No measured strategy applications. One incomplete correctness check.
All records retain draft status.

Proposed experiment

Choose an eligible strategy and check its remaining conditions.
Implement it, verify correctness, and measure total cost against its baseline.

Advisor discussion: which strategy and implementation should we evaluate first?

Backup: complete record inventory

Record kind	Count	What it stores
Application	1	GAPBS source and version
Kernel	6	Computation and correctness check
Implementation	8	Code, loops, and 77 access patterns
Input	4	Kronecker / uniform, scales 16 / 22
Machine	1	mbit10 hardware and CPU flags
Profile	32	Measurements for an implementation/input/machine
Strategy	5	Target, effects, conditions, reported benefit
Intrinsic	2	Instruction wrapper and required ISA

59 records validate. All are draft. 14 initial tickets and 7 extension tickets are resolved.

Backup: application and profile coverage

Kernel	Recorded implementations	Profiles
PageRank	Gauss–Seidel, Jacobi	8 complete
Breadth-first search	Direction-optimizing	4 complete
Betweenness centrality	Brandes	4 complete
Single-source shortest paths	Delta-stepping	4 complete
Connected components	Afforest, Shiloach–Vishkin	8 complete
Triangle counting	Ordered	3 complete, 1 incomplete

Each implementation uses 4 inputs: Kronecker and uniform random graphs, each at scales 16 and 22 with generator parameter $k = 16$.

Backup: implemented database operations

Capability	Implemented interface
Validate and store records	validate, add, build
Search access patterns and semantic facts	find, sql
Compare implementations of a kernel	implementations, including --applies
Screen strategy targets	strategies --pattern / --loop / --input
Export a workload view	view implementation input machine
Capture and profile the machine	capture-machine, profile
Refresh simulated cache metrics	recompute-cachegrind

Versioned YAML is the master copy. SQLite is generated for queries.

Backup: profiling and evidence types

Evidence	Recorded content	Interpretation
Correctness	Verifier outcome and timeout	31 complete profiles
Timing	1, 2, 4, 8, 16 threads, 5 trials	Measured application trial time
Memory footprint	Array sizes and footprint/L3	Recorded sizes and formulas
Index stream	Reuse, sequentiality, degree skew	Features in 12 profiles
Cachegrind	Kernel-function cache references/misses	Simulated, single-threaded
Bottleneck	Classification with its basis	Inferred, not a counter measurement

Profiles retain source/build identity, commands, raw-output locations, host load, thread binding, stage outcomes, and evidence references.

Backup: optimization strategies and intrinsics

Strategy	Target	Recorded effect
Packing	Access pattern	Indirect read becomes a stream, plus a packing pass
Software prefetch	Access pattern	Add a non-binding early access
SIMD gather	Access pattern	Widen lanes per access
Vertex reordering	Input	Reorder indices
Loop tiling	Loop	Restructure the loop

Intrinsic records: `_mm_prefetch` and `_mm512_i32gather_ps`

Strategy application links and derived ISA checks are implemented.

Backup: safeguards and remaining boundaries

Implemented safeguard	Remaining boundary
Schema, vocabulary, cross-record checks	Valid structure does not establish a research claim
Unknown semantics remain unknown	Strategy screening is not a full correctness proof
Derived ISA and compiler/machine checks	No strategy-applied implementation measured yet
Correctness status and incomplete profiles	Triangle counting / large Kronecker is incomplete
Evidence basis and raw-output references	Simulated cache misses and inferred bottlenecks
Workload-view generation	Live HW-agent integration not demonstrated

Measurement context: shared mbit10 host, one socket per run.

This update uses committed profile records, not a new measurement campaign.

Backup: PageRank timing and setup

Intel Xeon Gold 6326, one socket, 24 MiB L3, 5 trials at each thread count

Threads	Scale 16 median (ms)	Scale 22 median (ms)
1	20.45	2539.72
2	10.28	1281.29
4	5.28	646.16
8	5.18	335.84
16	5.20	190.22

Scale 16: 65,536 vertices, recorded footprint 8.33 MB

Scale 22: 4,194,302 vertices, recorded footprint 580.35 MB

Timing scope: application Trial Time. Raw runs date to 2026-09-22.